



MARCH 28, 2026

DIGITAL FORENSICS INVESTIGATION REPORT

Forensic Disk Image Analysis and File Recovery



Contents

1. Executive Summary 2

2. Tools Used 2

3. Methodology 3

4. Challenges Encountered 5

5. Validation and File Integrity 6

6. Recovered Files and Metadata 7

7. Conclusion 8

1. Executive Summary

This report presents the findings of a forensic examination of a binary disk image named 'carveit.' The objective was to recover deleted and inaccessible files using file signature analysis and binary carving. A Python script automated recovery, supported by manual hex inspection in HxD. In total, **22** candidate files were extracted, of which **17** were confirmed valid.

Category	Count	Note
Total files carved	22	JPG, PNG, PDF, GIF
Confirmed valid and readable	17	Passed
Excluded — internally corrupted	5	Valid boundaries; failed to render

Table 1 — Recovery summary

2. Tools Used

Tool	Version	Purpose
HxD Hex Editor	v2.5	Manual binary inspection; identified padding region and file header positions
forensic_carver.py	Python 3.10	Automated carving, signature scanning, MD5 hashing, hashes.txt output
Windows PowerShell	v7.6.0	Independent MD5 cross-verification via Get-FileHash
Windows Photos & PDF Reader	Built-in	Application-level rendering validation for all recovered files

Table 2 — Tools and their roles

3. Methodology

Inspection of the disk image in HxD revealed that the opening 116 KB consisted entirely of 0x20 space-byte padding, a deliberate anti-forensic wiping technique that destroyed all file system metadata. File signature analysis was employed: the Python script scanned the full binary stream **for JPEG (FF D8 FF / FF D9), PNG (89 50 4E 47 / IEND), PDF (%PDF / %%EOF), and GIF (GIF8 / 00 3B)** header and footer pairs, extracting byte ranges between each match. Per-type integrity checks, minimum size filters, and **MD5** hashing were applied before file was saved to. /max/.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000010	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000020	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000030	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000040	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000050	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000060	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000070	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000080	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000090	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000A0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000B0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000C0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000D0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000E0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000000F0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000100	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000110	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000120	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000130	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000140	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000150	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000160	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000170	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000180	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000190	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001A0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001B0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001C0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001D0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001E0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000001F0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000200	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000210	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000220	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000230	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000240	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000250	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000260	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000270	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000280	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000290	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002A0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002B0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002C0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002D0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002E0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
000002F0	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000300	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000310	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000320	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000330	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000340	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000350	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000360	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000370	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000380	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	
00000390	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	20	

Figure 1 — HxD: 0x20 padding confirming anti-forensic disk wiping

DIGITAL FORENSICS INVESTIGATION REPORT

```
forensic_carver.py X
252 print(f" Make sure 'carveit' is in the same folder as this script.")
253 print(f" Script is running from: {os.getcwd()}\n")
254 sys.exit(1)
255
256 # --- Create output folder ---
257 output_dir = last_name # folder named after last name
258 os.makedirs(output_dir, exist_ok=True)
259 print(f"\n Output folder: {os.path.abspath(output_dir)}")
260
261 # --- Load disk image ---
262 print(f" Loading disk image: {image_path} ...")
263 with open(image_path, "rb") as f:
264     image_data = f.read()
265
266 image_size = len(image_data)
267 print(f" Image size: {image_size:,} bytes ({image_size/1024/1024:.2f} MB)")
268
Shell X
Type : PNG
Start offset: 0x00DC7392 (14447506 bytes)
End offset: 0x00DE5ADF (14572255 bytes)
File size : 124,749 bytes (121.83 KB)
MD5 hash : A5F4A186FF91859B1914D3006E9F5C1D

[*] Searching for PDF files ...
Found 1 potential header(s).
[+] Carved: pdf_001.pdf
Type : PDF
Start offset: 0x00078ED3 (495315 bytes)
End offset : 0x00080ACC (527052 bytes)
File size : 31,737 bytes (30.99 KB)
MD5 hash : 9D5D42E5431E1F1249204824C53930F5

[*] Searching for GIF files ...
Found 1 potential header(s).
[+] Carved: gif_001.gif
Type : GIF
Start offset: 0x00D99C6C (14261356 bytes)
End offset : 0x00D9AE77 (14265975 bytes)
File size : 4,619 bytes (4.51 KB)
MD5 hash : CD50D5D005495D557130D320D12713C2

=====
CARVING COMPLETE - 22 file(s) recovered
=====

[✓] hashes.txt written -> max\hashes.txt

All recovered files saved to: ./max/
```

Figure 2 — Thonny: 22 files recovered and hashes.txt written

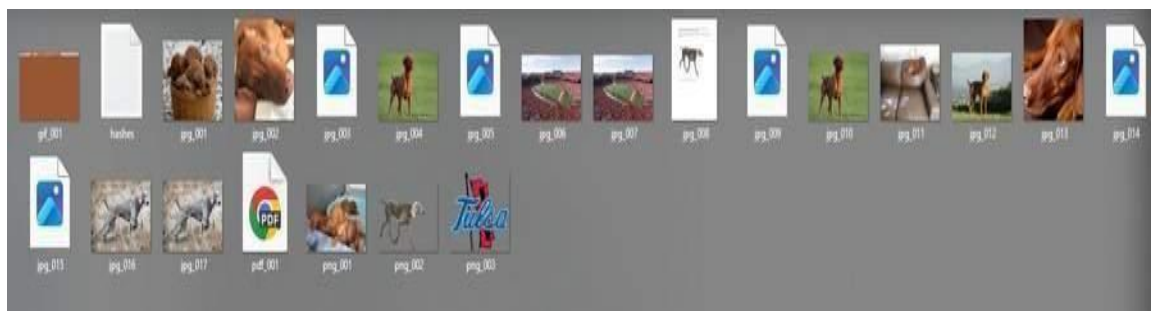


Figure 3 — Output folder with 17 valid recovered files

4. Challenges Encountered

Challenge	Description	Resolution
Anti-forensic Wiping	First 116 KB: 0x20 padding; no file system or partition table	Full binary stream scanned; first header at 0x0001CC08
False Positive Headers	EXIF thumbnails triggered nested FF D8 FF detections	Size thresholds and integrity checks eliminated false positives
Challenge	Description	Resolution
Corrupted File Content	5 JPEGs: valid boundaries but failed to open	Excluded post application-level validation;

Table 3 — Key challenges and resolutions

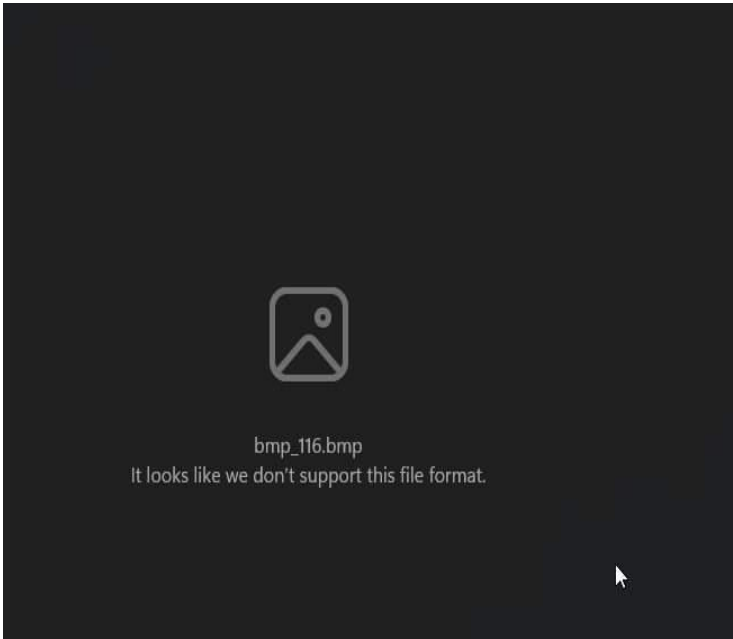


Figure 4 — Windows Photos: corrupted error for jpg_005.jpg (representative of 5 excluded files)

5. Validation and File Integrity

Integrity was confirmed through the three-layer process below. All **MD5 hashes** generated by the script were independently recomputed via PowerShell **Get-FileHash**, all **22** matched **hashes.txt** exactly.

Layer	Method	Outcome
Byte & Size Check	Header/footer verified; min size filter applied	Only complete files saved to disk
Application Rendering	All files opened in Windows Photos / PDF reader	17 valid; 5 corrupted JPEGs excluded
MD5 Verification	Script hash vs. PowerShell GetFileHash	All 22 hashes matched hashes.txt — 100%

Table 4 — Validation framework

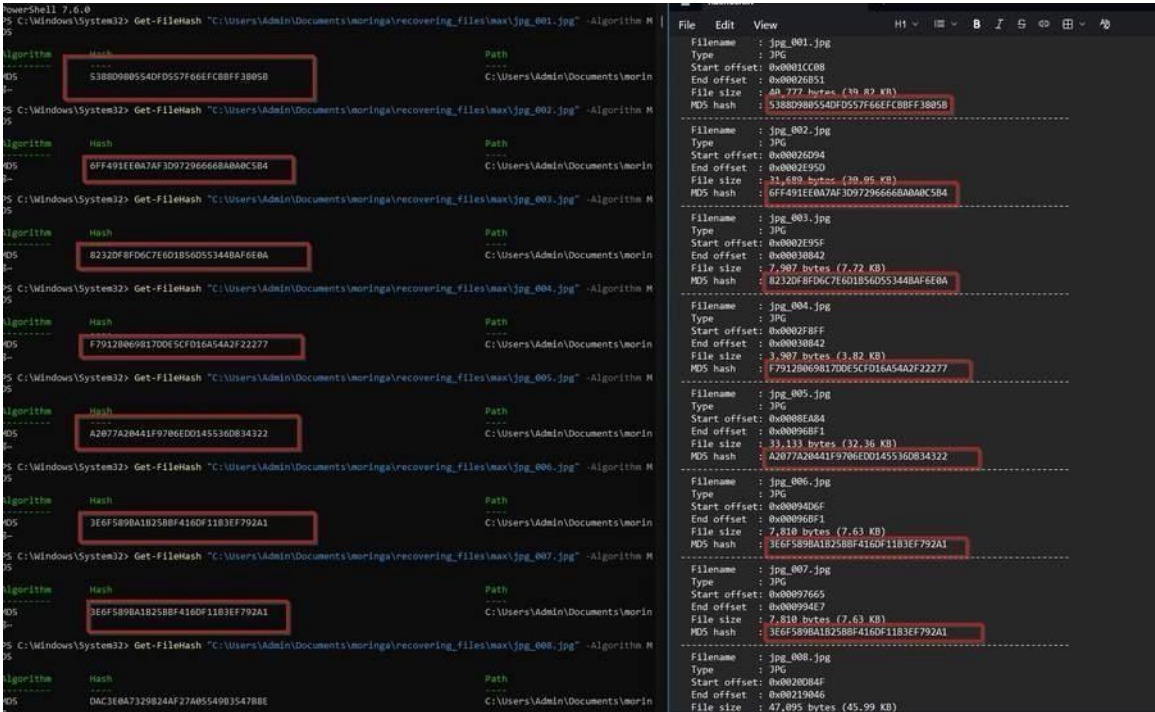


Figure 5 — PowerShell hashes matched against hashes.txt for all JPG files

DIGITAL FORENSICS INVESTIGATION REPORT

```

PS C:\Windows\System32> Get-FileHash "C:\Users\Admin\Documents\morina\recovering_files\max\png_002.png" -Algorithm MD5
Algorithm Hash Path
-----
MD5 879C3E60DF905E174ECE22DF184028C8 C:\Users\Admin\Documents\morina

PS C:\Windows\System32> Get-FileHash "C:\Users\Admin\Documents\morina\recovering_files\max\pdf_001.pdf" -Algorithm MD5
Algorithm Hash Path
-----
MD5 905D42E5431E1F1249204824C53938F5 C:\Users\Admin\Documents\morina

PS C:\Windows\System32> Get-FileHash "C:\Users\Admin\Documents\morina\recovering_files\max\png_003.png" -Algorithm MD5
Algorithm Hash Path
-----
MD5 ASF4A186FF91B59B1914D3006E9F5C1D C:\Users\Admin\Documents\morina

PS C:\Windows\System32> Get-FileHash "C:\Users\Admin\Documents\morina\recovering_files\gif_001.gif" -Algorithm MD5
Algorithm Hash Path
-----
MD5 6850D508C549505571288328B127136A C:\Users\Admin\Documents\morina

PS C:\Windows\System32>
  
```

```

-----
Filename : png_001.png
Type : PNG
Start offset : 0x000948A
End offset : 0x001DEF28
File size : 1,202,798 bytes (1174.61 KB)
MD5 hash : 56CA533EEAC8B8A5CCE923E17120809
-----
Filename : png_002.png
Type : PNG
Start offset : 0x001DFF8B
End offset : 0x002000C6
File size : 184,635 bytes (180.31 KB)
MD5 hash : 879C3E60DF905E174ECE22DF184028C8
-----
Filename : png_003.png
Type : PNG
Start offset : 0x00C7392
End offset : 0x00DE540E
File size : 124,749 bytes (121.83 KB)
MD5 hash : ASF4A186FF91B59B1914D3006E9F5C1D
-----
Filename : pdf_001.pdf
Type : PDF
Start offset : 0x00078ED3
End offset : 0x00080AC
File size : 31,737 bytes (30.99 KB)
MD5 hash : 905D42E5431E1F1249204824C53938F5
-----
Filename : gif_001.gif
Type : GIF
Start offset : 0x00099C6C
End offset : 0x0009AE77
File size : 4,610 bytes (4.51 KB)
MD5 hash : 6850D508C549505571288328B127136A
  
```

Figure 6 — Verification for PNG, PDF, and GIF files confirmed

6. Recovered Files and Metadata

Filename	Type	Size (KB)	MD5 Hash	Status
jpg_001.jpg	JPG	39.82	5388D980554DFD557F66EFCBBFF3805B	Valid
jpg_002.jpg	JPG	30.95	6FF491EE0A7AF3D97296666BA0A0C5B4	Valid
jpg_003.jpg	JPG	7.72	8232DF8FD6C7E6D1B56D55344BAF6E0A	Valid
jpg_004.jpg	JPG	3.82	F7912B069817DDE5CFD16A54A2F22277	Valid
jpg_006.jpg	JPG	7.63	3E6F589BA1B25BBF416DF11B3EF792A1	Valid
jpg_007.jpg	JPG	7.63	3E6F589BA1B25BBF416DF11B3EF792A1	Valid
jpg_008.jpg	JPG	45.99	DAC3E0A7329824AF27A05549B3547B8E	Valid
jpg_009.jpg	JPG	7.72	8232DF8FD6C7E6D1B56D55344BAF6E0A	Corrupted
jpg_010.jpg	JPG	3.82	F7912B069817DDE5CFD16A54A2F22277	Corrupted
jpg_011.jpg	JPG	7.93	F62D8E9CB1C9183312B96A5E0389E0BE	Valid

jpg_012.jpg	JPG	24.08	A3A853BBA0EC2EF6F9F15D1A01D863C1	Valid
jpg_013.jpg	JPG	457.44	2DADF2A77B8196DE6129DB13D5075EF1	Valid
jpg_014.jpg	JPG	7.73	346C3BABE42A42CEC3E76613341B7F07	Corrupted
jpg_015.jpg	JPG	7.53	035CC1B8EAADC89E47C4642F9C41FC8F	Corrupted
jpg_016.jpg	JPG	7.18	D11A5FD13522B42259451673E4290790	Valid
jpg_017.jpg	JPG	7.18	D11A5FD13522B42259451673E4290790	Valid
png_001.png	PNG	1174.61	56CA533EEAC8B8A5CCBE923E17120809	Valid
png_002.png	PNG	180.31	B79C3E60DF905E174ECE22DF184028C8	Valid
png_003.png	PNG	121.83	A5F4A186FF91859B1914D3006E9F5C1D	Valid
pdf_001.pdf	PDF	30.99	9D5D42E5431E1F1249204824C53930F5	Valid
gif_001.gif	GIF	4.51	6B50D5D8C5495D557128B328B127136A	Valid

Table 5 — Metadata for all 22 carved files (green = valid; red = corrupted)

7. Conclusion

This investigation recovered 17 valid, **13 JPEG, 3 PNG, 1 PDF, and 1 GIF** from a corrupted disk image. The results demonstrate that multi-layer validation is essential in file carving workflows, as structural byte integrity alone cannot confirm file completeness. Future work would incorporate Autopsy for improved fragmentation handling.

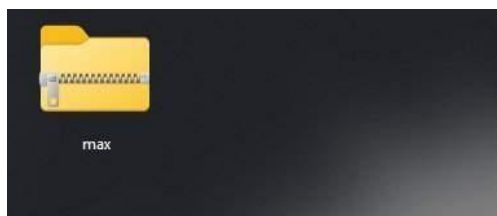


Figure 7 — max.zip containing all 17 valid recovered files

